Konstantin Dolbilov

Full-stack Web Developer

Gijon, Spain (UTC+2) • cossamedia@gmail.com • github.com/konfrontend • linkedin.com/in/konstantin-dolbilov-94720b361

Full-stack web developer with 7+ years shipping TypeScript products from UX to database: Vue.js and React dashboards, Spring Boot and Node.js REST APIs, and 3D tools in WebGL. Open to contract or permanent roles.

SKILLS

- Languages: TypeScript, JavaScript, Java, SQL, HTML, CSS
- Frontend: React, Vue.js, Zustand, TanStack Query, Tailwind CSS, shadcn/ui, Vite, Web Workers, WebSockets
- Graphics: three.js, React Three Fiber, WebGL, GLSL, SVG, HTML5 Canvas, Computational Geometry
- Backend: Node.js, Hono, Spring Boot, REST API, MCP, PostgreSQL, SQLite, MariaDB, Drizzle ORM, Zod
- Testing and DevOps: Vitest, Playwright, JUnit, GitHub Actions, CI/CD, Docker, Electron, Turborepo
- AI tooling: Claude Code, Agentic Workflows, Prompt Engineering, Architecture Decision Records

PROFESSIONAL EXPERIENCE

Loadster – Cloud SaaS for load testing and uptime monitoring of websites and APIs.
Full-stack Web Developer Mar 2019 – Present

- Shipped Playwright scripting end to end, from the Spring Boot REST API and runtime to live traces of a running load test and a Monaco Editor with custom IntelliSense.
- Rewrote the Loadster Recorder extension in TypeScript on the WebExtension API and Chrome DevTools Protocol, generating Playwright Test code live in Chrome and Firefox; authored about half of its commits.
- Built the Loadster MCP server in Spring Boot: 23 tools that let AI agents drive the whole product, with 376 of the package's 419 JUnit test cases.
- Built monitoring full stack, from Flyway migrations on MariaDB to the Vue.js dashboard: notification policies across timezones, period graphs, monitor state over WebSockets, and public status pages.
- Optimized Vue.js dashboard performance at scale: custom HTML5 Canvas charts for real-time metrics, virtualized editors holding thousands of rows, and 100,000-row trace reports loaded in parallel pages.
- Rebuilt the engineering environment for AI agents and introduced Vitest to the dashboard and Node.js runtimes: merged pull requests rose from about 30 to 55 a month, team test cases from about 500 to 1,900.

Jigawatt Solar – Software suite for solar installers: interactive 3D design and energy modeling.
Senior Front-end Developer Oct 2019 – Jul 2026

- Authored the 3D roof modeling engine in React Three Fiber and WebGL: procedural volume generation, DCEL-based partitioning and realtime editing on a persisted topology graph, covered by 64 Vitest cases.
- Built the shade-loss simulation in Web Workers with OffscreenCanvas: sun-path raycasting over BVH trees measures how much energy each roof loses to shade while the editor stays responsive.
- Decided the production frontend stack alone: React, TypeScript, Vite, Vitest, Zustand and three.js; authored 47% of the 3D designer's commits, all 17 test files (140 cases) and 11 ADRs.
- Built in-browser proposal and contract signing on PDF.js and pdf-lib: AcroForm fields prefilled by shortcodes, locked per party on submit, and flattened with embedded signatures into the signed file.

PROJECTS

autotriage – Claude Code plugin that triages a freelance job posting and writes the proposal letter.
Author & Sole Developer Sep 2026 – Present

- Authored a public Claude Code plugin that takes a freelance job posting through an agentic workflow, one skill per state, to a proposal letter ready to paste, on a catalogue of 23 risk flags and 40 checks.
- Enforced the workflow with hooks that deny an out-of-order step and validate every data write against its JSON Schema with Ajv, plus a verify script of 28 checks covered by 18 tests on the Node.js test runner.

Time Stop – Local-first time tracker app with an optional self-hosted sync server.
Author & Sole Full-stack Developer Aug 2026 – Present

- Authored the Electron desktop app in React, TanStack Query and shadcn/ui, 70 pull requests in five weeks solo with AI agents, shipped for macOS and Windows in two public releases under MIT.
- Designed the local-first model, SQLite as the source of truth with an append-only change log, and built the Hono on Node.js sync server with PostgreSQL that mirrors it one way, idempotent on replay.
- Designed the two-dialect database layer in Drizzle ORM and Zod with compile-time and snapshot drift checks, and chose SQLite at about 52 MB of memory over PGlite at about 1.3 GB in an ADR.
- Built GitHub Actions CI and releases: Vitest against a PostgreSQL service container, Playwright end-to-end tests of the Electron app, and one git tag shipping both desktop builds and a multi-arch Docker image.